

A Comparative Study of PostgreSQL and MongoDB in a Banking Application

Aidan Ryther
Rochester Institute of Technology
arr9180@g.rit.edu

Tiffany Lee
Rochester Institute of Technology
tl5110@rit.edu

Vivian Hernandez
Rochester Institute of Technology
vah7365@rit.edu

ABSTRACT

We evaluate SQL and NoSQL for a small banking application by implementing the same features with PostgreSQL and MongoDB Community Server. The application supports a ton of different features like opening accounts, depositing, withdrawing, transferring, balancing checks, and a view of your transactions. Both systems run with ACID transactions and durable writes. We compare three things. For development, we track how long each feature takes and how much code touches the database, plus any changes to schemas or indexes. For query complexity, we count joins in SQL and note how involved the equivalent MongoDB queries are. We measure performance by looking at throughput and response time while running a fixed mix of reads and writes under different levels of concurrency on the same machine with the same setup. Using this banking application, our goal is to see whether SQL or NoSQL is the better choice when strong consistency is prioritized over availability, and to point out tradeoffs that go beyond this single case.

KEYWORDS

Banking, PostgreSQL, MongoDB, SQL, NoSQL, ACID Transactions, OLTP, Performance

1 INTRODUCTION

Retail banking puts correctness ahead of availability. Balances must be exact, transfers committed as one unit, and two concurrent withdrawals from the same account should not both succeed. This paper looks at how a relational model and a document model handle that basic workload. We build the same small banking service twice: once on PostgreSQL and once on MongoDB Community, both configured for ACID transactions and durable writes. Now to keep the runs fair and repeatable, we follow the benchmarking best practices that have been thoroughly tested in previous experiments [10, 15]. To keep these experiments grounded, we'll mainly be comparing the real world results.

In comparing SQL and NoSQL there are too many tradeoffs for one paper, so we keep the scope tight around a concrete system design problem. A lot of online systems are built to keep running smoothly by prioritizing availability, even if that means relying on eventual consistency. Banking is the opposite. Money movements must be exact, so we use a small banking app to see how each model handles strict correctness. PostgreSQL is our relational baseline and MongoDB Community stands in for a document store. We hold hardware, dataset size, and index plans fixed so that the

only differences we see come from the data model and transaction behavior, not doing trickery with the settings [1, 10, 15].

We picked PostgreSQL because it is widely used and supports ACID transactions. It has real constraints, good indexing, and it's simple to run on one machine. We chose MongoDB Community Server because it's very easy to set up, one of the most popular choices with programmers, allows us to do multi-document transactions, and gives a flexible schema when you need to have flexible schemas. Both are free to run self hosted, so we can keep everything on the same box and control the settings. That lets us line up indexes, write durability, and transaction settings so the runs are fair. The data maps cleanly in both cases too. We have accounts and transactions, with keys, balances, and history. With this setup, any differences we see should come from the data model and query style, not from service [1].

From this experiment, we want to figure out which database fits better for any system that values consistency over availability, such as our bank app. You will see the same features built two ways, one in PostgreSQL and one in MongoDB, and what that teaches us. With PostgreSQL we keep accounts and transactions in related tables with keys and checks, and queries join them to move money and show recent activity. In MongoDB, we store data in documents, sometimes embedding recent activity with the account and sometimes keeping it in a separate collection and referencing it. These choices will shape the indexes we build, the way reads and writes are handled, and how much of the code has to interact with storage. We follow a simple test plan. First, increase concurrent users, then report latency and throughput so the results can be checked and repeated [15]. By the end, you should have a clear view of the tradeoffs you would feel in practice, including developer effort, query shape, and speed, so you can judge when SQL or NoSQL makes more sense for a correctness first app.

At a high level, we compare three areas. First, development, which is recording the time it takes each feature to build and how much code touches the database [3]. Second, query shape. In PostgreSQL that means joins, constraints, and a fixed schema. In MongoDB that means reading and updating documents, which sometimes have references in them, and a flexible schema [1]. We also see how these choices line up with common SQL versus NoSQL tradeoffs, particularly taking note of which indexes each one needs [1]. Third, their performance, or runtime behaviors as we add concurrent users. Using the same machine we report their throughput and response times, following basic benchmarking guidance to avoid common traps [10, 15]. At the same time we enforce the banking rules (atomic transfers, no negative balances), but we do not rate security features or admin tools. The goal is to show the costs you feel while building and what happens when the app is busy so you can map the results to your own case.

2 PLAN TO EVALUATE

For our evaluation, we maintain the same banking rules no matter which database we use. Each account has an id and a balance that cannot go below zero unless an overdraft is on. When first opening an account, the user starts at zero. Deposits and withdrawals are performed in a single transaction. A transfer debits one account and credits the other as one unit, and we tag each transfer with an external id so a retry does not double charge. The transaction log is appended only with time, amount, and account ids, and reads only show committed data. On the database side, we turn on ACID transactions and durable writes in both systems. In PostgreSQL we rely on keys and checks, and wrap money moves in a transaction. In MongoDB, we use multi-document transactions with majority writes and snapshot-style reads. We keep the index plan aligned in intent on both builds, so queries follow the same access paths even if the definitions differ. The rest of the setup stays fixed, too. This means that when testing, we plan to use the same machine, same client mix, same pool sizes, a short warm-up, and timed runs. These choices follow basic guidelines on fair testing and avoiding configuration tricks [10, 15].

2.1 Data Models and Indexes

In PostgreSQL we use two tables:

- `accounts(id, balance, overdraft, ...)`
- `transactions(id, account_id, amount, type, transfer_id, ...)`

PostgreSQL primary and foreign keys allow us to tie rows together, and we use CHECK constraints to handle simple rules like non-negative amounts. When we move money between two accounts, we save two records with the same `transfer_id`, one for the debit and one for the credit. They're saved together in a single transaction, so either both are written or neither is, which prevents half-finished transfers. To improve speed, we use a composite B-tree index on recent activity per account (`account_id, ts DESC`) and a unique index on `transfer_id` to prevent duplicates [13, 14].

In MongoDB we mirror that shape with two collections:

- `accounts`
- `transactions`

Account documents hold the current balance and flags. Transaction documents store `{ accountId, ts, amount, type, transferId, ... }`. Reads and writes work on whole documents, and when we need a link we keep `accountId`. We create a compound index `{ accountId: 1, ts: -1 }` for "recent activity per account" and a unique index on `{ transferId: 1 }` to stop a double post [11, 12]. The access paths line up in intent across both systems even though the definitions are different.

2.2 Workload mix, dataset size, and concurrency levels

For this project, we will design a workload that mirrors operations of a real banking system. The operations include opening new accounts, deposits, withdrawals, transfers, balance checks, and viewing recent transactions. Deposits and withdrawals operations will be the most frequent, followed by balance checks and transfers. Account creation will be less common. Transfers will test whether each system can handle multi-record transactions correctly.

The dataset will consist of approximately 100,000 accounts and approximately 1 million transactions. This size is large enough to mirror real-world banking systems while remaining feasible on a single machine. To evaluate concurrency handling, the workload will be executed with different numbers of clients (1, 5, 10, 20, and 50) running simultaneously. This variation will allow us to observe how each system performs under both light and heavy concurrent loads.

2.3 Metrics to Report and How We Record Them

We will measure three categories of metrics to provide a balanced evaluation.

- **Development effort:** measured by the time it takes to build each feature, how many lines of code directly work with the database, and how many schema or index changes are required.
- **Query and model complexity:** measured by the number of joins in PostgreSQL queries, the complexity of MongoDB aggregation pipelines or transactions, and a qualitative look at how readable and maintainable the queries are.
- **Performance:** measured by transactions per second and response latency at different concurrency levels. Each test will be run several times and the results will be averaged to smooth out the random variation.

Altogether these three areas let us see the tradeoffs between ease of development, complexity of queries, and runtime performance.

2.4 Fairness Rules, Hardware, and Run Procedure

As mentioned earlier, PostgreSQL and MongoDB both run on the same machine to keep things fair. This machine will specifically have a 4-core CPU with 16GB RAM and SSD storage. Each will be set up with durable writes and ACID transactions turned on. We'll use equivalent indexing strategies so neither database has an edge, and we'll avoid system-specific optimizations that could tilt the results toward one side.

The procedure will be consistent across systems. First, the dataset will be loaded into the database. Then, a warm-up phase with preliminary queries will be executed to bring caches and buffers into a steady state. Finally, the workload will be run at differing concurrency levels (1, 5, 10, 20, 50 clients). Each experiment will be repeated several times, and the average results will be reported to improve reliability.

2.5 Limits and Threats to Validity

There are a few limits we should acknowledge. The workload and dataset are simpler than a real banking system and don't cover advanced features. Both databases are tested on a single node, so results may not represent distributed setups like PostgreSQL replication or MongoDB sharding, which would matter for scalability and availability. Measuring development effort is also partly subjective and may depend on the developer's background with SQL or MongoDB since familiarity could affect how quickly features are implemented and how complex the code ends up being.

3 LITERATURE REVIEW

Modern retail banking is an online transaction processing workload that must be correct first and fast second. Account balances must always reflect all committed transfers, and no double debit or phantom credit can appear just because two clients hit the system at the same time. Prior work on data consistency in distributed NoSQL platforms shows that weaker consistency levels and looser isolation often improve throughput and latency in geo-replicated deployments, but they do so by allowing different replicas to temporarily observe different states [8]. That kind of divergence may be acceptable for analytics-style or content-serving applications, but not for money movement, where even a short window of disagreement between two reads is a problem. At the same time, application teams still care about developer effort, deployment flexibility, and scaling cost. That tension between strict correctness and flexibility sits at the core of the ongoing SQL versus NoSQL discussion [1, 5, 6, 9].

Relational databases achieve correctness with fixed schemas, declarative constraints, and ACID transactions that have been hardened for decades. PostgreSQL is a representative system in that space. It supports multi-statement transactions, foreign keys, uniqueness constraints, and developer-defined CHECK constraints, and it lets developers create targeted multicolumn B-tree indexes to match common access paths [13, 14, 16]. Document databases originally came from the other end of the design space. Systems like MongoDB focused on letting teams move faster: store JSON-like documents directly, avoid rigid schemas, and scale out horizontally. Over time, those systems have added stronger guarantees, including multi-document transactions and durability options intended to prevent lost writes under concurrency [5, 9, 11, 12, 17].

Because these stacks grew out of different assumptions, a meaningful comparison is not “run one query and time it.” The database benchmarking community has repeatedly shown that naïve microbenchmarks are easy to bias, and that results can flip depending on index design, workload mix, concurrency level, durability settings, and cache state [2, 4, 10, 15, 18]. What actually matters is whether each system can enforce banking-style correctness, how hard it is for developers to get there, and what performance you get once you turn the safety knobs all the way up.

Our project is organized around four themes that show up across the literature: (1) why strong consistency matters for transactional, banking-style workloads, (2) how relational databases like PostgreSQL enforce correctness, (3) how modern document stores like MongoDB approach the same operations, and (4) what a fair, reproducible SQL-vs-NoSQL comparison looks like. We also connect these technical questions to engineering cost, because development effort is itself a real constraint in production systems [3].

3.1 Strong consistency in transactional banking systems

Money movement is the textbook case for requiring strict consistency. A transfer should either debit one account and credit the other, or not happen at all, even if two clients try to move overlapping funds at the same time. Ferreira et al. study how different consistency levels behave in cloud-hosted NoSQL systems under realistic, data-intensive workloads [8]. They show that relaxing

consistency (for example, letting replicas acknowledge writes before the full cluster is synchronized) can boost throughput and reduce observed latency, but at the cost of replicas temporarily disagreeing about the “true” value of shared data [8]. In a bank, that window is not tolerable: it risks transiently overdrawing an account, double-posting a transfer, or producing an audit trail that cannot be defended later.

Historically, this requirement pushed transactional workloads toward relational engines. The traditional view was that document stores could not provide the same guarantees once a logical operation touched more than one row, or more than one document. That gap has narrowed. Schultz and Demirbas analyze MongoDB’s distributed transaction protocol and give a modular proof argument that MongoDB’s multi-document transactions satisfy key safety properties such as atomic commit and consistent visibility across replicas [17]. In practice, this means MongoDB can now execute a transfer like “withdraw \$X from account A and deposit \$X into account B” inside a single ACID-style transaction with majority write concern, so either both updates commit or neither does [11, 17].

From an application point of view, the two stacks are now closer than they used to be. PostgreSQL has long provided multi-statement ACID transactions with snapshot-style isolation on a single node, so readers inside a transaction see a consistent view of the database while updates are applied atomically. MongoDB now also exposes snapshot-style reads inside a multi-document transaction, and lets clients choose majority durability so a write is only acknowledged after it has been replicated to a voting majority, improving survivability if one node fails [11, 17]. The open question is no longer “can the database enforce correctness at all,” but “what does it cost in performance, and how easy is it for developers to actually use those guarantees correctly in code.”

3.2 Relational databases for correctness:

PostgreSQL

PostgreSQL represents the classic relational model: correctness is enforced before the application code even runs. Foreign keys connect related rows and prevent orphans. CHECK constraints block illegal values such as negative balances. UNIQUE constraints prevent accidental reuse of a transfer identifier [14]. Every set of related updates is wrapped in a transaction that either fully commits or rolls back. On top of that, PostgreSQL exposes multicolumn B-tree indexes, which let developers match the physical access path to the logical query pattern (for example, “recent activity for this account, newest first”) [13]. Those indexes, plus constraints, can often express business rules declaratively instead of pushing them into application code.

Relational engines are also still very competitive in online transaction processing (OLTP) when the schema is known and tuned. Salunke and Ouda benchmark PostgreSQL and MySQL under common transactional operations such as inserts, updates, and joins, reporting differences in throughput and latency and tying them back to each system’s storage and concurrency behavior [16]. Taipalus surveys database performance comparison studies and finds that the absolute “winner” changes from paper to paper, but one pattern is stable: index design, schema design, and tuning dominate headline throughput numbers [18]. In other words, relational systems

can still deliver high performance and predictable transactional integrity, as long as the data model is well understood.

There is also an engineering cost angle. A relational schema forces teams to design keys, constraints, and relationships up front. That can slow early iteration, but it gives you built-in guards once the system is live. Prior work on software development effort shows that unplanned complexity and rework translate directly into more engineering hours [3]. In a banking-style workload, that trade can actually reduce long-term effort: PostgreSQL will reject invalid state at the database layer instead of relying on every caller to remember to check it [13, 14, 16].

3.3 Document databases for operational data: MongoDB

Document databases were originally pitched as a better fit for fast-moving product teams [9]. Instead of normalizing data across tables, the application can store the fields it needs in one document. Reads can then pull a full view of an entity without doing joins, which simplifies code and can cut down on round trips [1, 5, 9, 11]. MongoDB supports compound indexes like `{accountId: 1, ts: -1}` to answer “recent activity for this account” efficiently, and it can enforce unique indexes on application-supplied identifiers to stop the same transfer from being posted twice [11, 12]. Together, flexible documents plus targeted indexes let teams shape the data model around how the app is actually queried, instead of designing a full relational schema up front.

Recent studies have profiled document stores under operational, not just analytics, workloads. Carvalho et al. compare several document databases, including MongoDB, and evaluate how they behave under different read/write mixes [5]. They show that observed latency and throughput depend heavily on access pattern, configuration, and indexing strategy [5]. Györfi et al. compare MongoDB to a document-oriented mode in MySQL aimed at big-data style ingestion and retrieval, and report concrete tradeoffs in query speed versus model flexibility [9]. Carey et al. propose CH2, a benchmark meant to stress both operational and analytical style access on NoSQL systems at the same time, arguing that modern applications tend to blur that line anyway [4].

Benchmarking has also started to move past single-document CRUD tests. Beckermann extends the well-known YCSB workload to include multi-operation transactions with ACID guarantees, so NoSQL systems can be evaluated under workloads that look more like real OLTP sessions instead of isolated key-value lookups [2]. This matters for our project: newer NoSQL evaluations are explicitly asking systems like MongoDB to handle the same “debit, credit, show me recent history” patterns that a relational database would run [2, 4, 5, 9, 17], rather than treating MongoDB as an eventually consistent log where correctness is someone else’s problem.

3.4 Fair comparison of SQL and NoSQL systems

Publishing “database X is faster than database Y” without context is considered bad form. Raasveldt et al. catalog common traps in database benchmarking: comparing systems with different durability settings, timing unrealistic microqueries, using workloads that only favor one engine, or hiding warm-up effects [15]. Keep and

Ingo give similar guidance for MongoDB: fix the hardware, load realistic data, warm caches before timing, and clearly document read and write concern so others can reproduce the run [10]. Taipalus echoes this in a systematic review, noting that many published comparisons are hard to interpret because they skip tuning details or silently mix analytical and transactional queries in the same reported number [18].

The community is also moving toward more transparent and workload-aware benchmarks. Defrance et al. argue for benchmark frameworks that are adaptable and explicitly document scoring assumptions so results can be interpreted in context instead of treated as universal truth [7]. CH2 and Transactional YCSB follow the same spirit for databases by encoding mixed workloads, realistic concurrency, and multi-statement transactions directly into the benchmark suites [2, 4]. Corovčák and Koupil compare six SQL and NoSQL systems side by side and conclude that the “best” choice depends on access pattern, consistency requirements, and scale assumptions, not on a single throughput number [6].

We align our own study with that guidance. We run PostgreSQL and MongoDB on the same hardware, enable durable writes and ACID-style transactions in both, align index intent, and drive a workload that looks like a small retail bank: open account, deposit, withdraw, transfer, check balance, view recent activity [2, 4, 6, 10–15]. We also plan to report development effort in addition to runtime metrics, because cost to build and maintain is part of the real tradeoff teams face when they decide between a relational model and a document model [1, 3].

4 PROGRESS REPORT

At this point in the project, we have created the basic scaffolding we need to compare PostgreSQL and MongoDB in a systematic way, and we have implemented the main banking workflow against MongoDB. We have also prepared data sets, kicked off both code paths, and described how we will load and exercise the system. The work completed so far falls into three categories. First, data. We generated and organized multiple realistic banking data sets under `banking_datasets/`, and we store each scenario in both CSV and JSONL form so that PostgreSQL and MongoDB will receive identical inputs. Second, application logic. We implemented account lifecycle and money movement operations for the MongoDB path, including deposits, withdrawals, transfers, balance checks, and basic recent activity reporting. Third, evaluation scaffolding. We created helpers to load data, verify it, and run manual sequences of operations so that later on we can time those operations under concurrency instead of just writing throwaway scripts.

We have not completed the PostgreSQL path, but the path itself is shaped out. The PostgreSQL code already has connection to a dedicated `CSCI_725_Project` database through `psycpg`, and the dataset was laid out with the design so that CSVs can be bulk loaded into relational tables with keys and constraints. The next work on that side is to solidify schema creation, constraints, and transactional behavior for money movement to bring it to feature parity with the MongoDB version. Once that is complete we will be able to run the same account open, deposit, withdraw, transfer, balance, and transaction history operations on both systems in a repeatable

way and begin collecting the performance and development effort metrics described in our plan.

4.1 PostgreSQL

The PostgreSQL track will be established as the relational baseline of the study. We have implemented a connection helper in `sql_implementation.py` that initiates a session to a local instance of PostgreSQL named `CSCI_725_Project` using `psycpg`. That provides us with a stable base to incrementally build the rest of the functions. We intend to replicate the same four logical entities that are modeled in the MongoDB build: customers, accounts, transactions, and merchants. The CSVs in `banking_datasets/` have been structured for that purpose. For instance, the files `baseline_customers.csv`, `baseline_accounts.csv`, and `baseline_transactions.csv` hold the same fields that are captured in the JSONL files that drive MongoDB. That will make it possible to load both engines with comparable data rather than generating two unrelated worlds.

The PostgreSQL schema will be designed according to the rules outlined in the plan. `accounts` will contain each account's id, customer id, balance in cents, overdraft limit, status, and timestamps. `transactions` will store individual movements, with an account id, a timestamp, a signed amount in cents, a type label (deposit, withdraw, transfer_debit, or transfer_credit), and a transfer_id so the debit and credit side of the same transfer can be associated. `customers` and `merchants` will contain stable information about the account holder and, if applicable, the counterparty. We will use foreign keys to ensure referential integrity, so that a transaction cannot point to a non-existent account. We will also add CHECK constraints on balances and amounts, as well as a UNIQUE constraint on transfer_id to prevent duplicate posting of the same transfer (as we will also check for this when evaluating correctness) [13, 14].

PostgreSQL's expected query plans also align well with our evaluation criteria. We can create multicolumn B-tree indexes, say on (`account_id`, `timestamp DESC`) in `transactions` to efficiently support queries like "show recent activity for this account" [13]. All money-moving operations (deposit, withdraw, transfer) will be executed in an explicit SQL transaction. For a transfer, that means subtracting from the source balance, adding to the destination balance, and inserting the corresponding debit and credit rows in `transactions`, and then committing or rolling back all of those actions as a single unit. These decisions let PostgreSQL serve as the reference for strong consistency and constraint-driven correctness, which is how it is positioned in the literature [13, 14, 16, 18].

The remaining PostgreSQL work is implementation work. We need to write the table creation DDL with constraints, the bulk loader that ingests the CSVs using COPY, and each banking operation in Python using `psycpg` so that it can be invoked and timed just like the MongoDB version. With that complete we will be able to measure not only runtime performance, but also development effort, which means lines of code that directly interact with storage and how many schema or index changes are required when we add features [3]. That ties directly back to one of our research questions.

4.2 MongoDB

Our MongoDB track is farther along. We have a working banking service running in `nosql_implementation.py` against a local MongoDB instance and database named `CSCI_725_Project`. The code creates four collections: `Customers`, `Accounts`, `Transactions`, and `Merchants`. Each collection is created with a JSON Schema validator that sets basic required fields, value types, and shape. For example, `Accounts` requires an integer `_id`, a customer_id, an integer balance stored in cents, an overdraft_limit, a status, and both creation and update timestamps. `Transactions` requires an integer primary id, an account_id, an integer amount in cents, a timestamp, and a type string such as deposit, withdraw, transfer_debit, or transfer_credit. It also allows a shared transfer_id, an optional merchant_id, and a note. This gives us a document model that is still flexible, but that already encodes business assumptions that we care about, like "an account must exist and have a balance" and "a transaction must record what happened" [9, 11, 12, 17].

We also implemented a data loader that can fill those collections with any of the prepackaged scenarios in `banking_datasets/`: `baseline`, `edgescases`, `hotspot`, and `payday`. Each one comes with CSV and JSONL files. The loader is built to read the JSONL ones. It normalizes timestamps into Python `datetime` objects, in UTC time. It maps the source field names into our chosen schema and inserts those documents into the proper collection. Balances and transaction amounts are stored as integer cents, rather than as floats. That's how real financial systems normally represent money, and it also avoids rounding drift. We also implemented a `verifyData` helper that prints out collection sizes and sample documents after loading. That lets us be sure load procedures are correct, before proceeding to measure queries at scale.

We then added the core banking operations on top of the collections and loader. The code can open new accounts for random customers, or reopen an account that is currently marked closed. It can deposit funds, withdraw funds with sufficient funds checks, and perform transfers between two accounts. A transfer is recorded as two transactions that share a generated transfer_id. One is a debit from the source account and one is a credit to the destination account. Both records carry the same channel and optional merchant information if the transfer is tied to a merchant payment. After adjusting balances with atomic `$inc` updates, the function inserts both transaction documents. This matches the behavior we described in the plan where a transfer is supposed to be one logical unit that moves money out of one account and into another. We also implemented balance queries, recent activity lookup for a given account (ordered by timestamp descending and limited to the most recent 10 entries), account close, account reopen, simple cleanup utilities, and basic pretty printing for console inspection.

This makes MongoDB usable for manual exploration. We can load realistic data, execute representative banking operations, and see the resulting state. It also provides hooks to reason about how easy it is to express banking rules in a document database. For instance, our current implementation does not permit overdraft in Python before applying a withdrawal or transfer, and it pairs credit and debit rows with the same transfer_id which is consistent with our earlier statement about application-layer enforcement of correctness. Next steps along the MongoDB path are to begin

leveraging stronger features from both the literature and from the official documentation. That includes creating a unique index on `transfer_id` so that the same transfer cannot be posted twice, creating a compound index on `account_id` and `timestamp` so that recent history queries scale, and pushing paired writes like transfers into an explicit multi-document transaction with majority write concern [11, 12, 17]. Making those changes will let us run MongoDB with ACID-style guarantees during benchmarking, which is important if we want to claim that both systems are being asked to solve the same banking problem rather than two different problems.

In general, the MongoDB side is playing the role of our first working implementation of the banking workload. The PostgreSQL side is being developed to match that same behavior under strict relational constraints. With both complete, we will be able to run on the same machine with the same slices of a dataset, drive concurrent clients, and measure throughput, latency, and development effort in the way described in our evaluation plan [3, 10, 15].

REFERENCES

- [1] Astera Analytics Team. 2025. SQL vs. NoSQL: 5 Main Differences. Astera Knowledge Center blog. Retrieved 2025-10-01 from <https://www.astera.com/knowledge-center/sql-vs-nosql/>
- [2] Benedikt Maria Beckermann. 2025. Transactional YCSB: Benchmarking ACID-Compliant NoSQL Systems with Multi-Operation Transactions. In *Datenbanksysteme für Business, Technologie und Web (BTW 2025) (Lecture Notes in Informatics)*. Gesellschaft für Informatik, Bonn, Germany, 1019–1030. doi:10.18420/BTW2025-67
- [3] Carlos Eduardo Carbonera, Kleinner Farias, and Vinicius Bischoff. 2020. Software development effort estimation: a systematic mapping study. *IET Software* 14, 4 (2020), 328–344. doi:10.1049/iet-sen.2018.5334
- [4] Michael J. Carey, Dmitry Lychagin, M. Muralikrishna, Vijay Sarathy, and Till Westmann. 2022. CH2: A Hybrid Operational/Analytical Processing Benchmark for NoSQL. In *Performance Evaluation and Benchmarking: 13th TPC Technology Conference (TPCTC 2021) (Lecture Notes in Computer Science, Vol. 13169)*. Springer, 62–80. doi:10.1007/978-3-030-94437-7_5
- [5] Inês Carvalho, Filipe Sá, and Jorge Bernardino. 2023. Performance Evaluation of NoSQL Document Databases: Couchbase, CouchDB, and MongoDB. *Algorithms* 16, 2 (Feb. 2023), 78. doi:10.3390/a16020078
- [6] Martin Čorovčák and Pavel Koupil. 2025. SQL vs NoSQL: Six Systems Compared. In *Proceedings of the 20th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2025)*. SCITEPRESS – Science and Technology Publications, Setúbal, Portugal, 41–53. doi:10.5220/0013217300003928
- [7] MaryBeth Defrance, Maarten Buyt, and Tijl De Bie. 2024. ABCFair: an Adaptable Benchmark Approach for Comparing Fairness Methods. In *Proceedings of the 38th Conference on Neural Information Processing Systems (NeurIPS 2024), Datasets and Benchmarks Track*. <https://openreview.net/forum?id=ByknnPI5Km> Poster; last modified 2025-01-17.
- [8] Saulo Ferreira, Júlio Mendonça, Bruno Nogueira, Willy Tiengo, and Ermeson Andrade. 2024. Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications. *Journal of Cloud Computing: Advances, Systems and Applications* 13 (Nov. 2024), 158. doi:10.1186/s13677-024-00716-7
- [9] Cornelia A. Györfi, Diana V. Dumșe-Burescu, Doina R. Zmaranda, and Robert Ș. Györfi. 2022. A Comparative Study of MongoDB and Document-Based MySQL for Big Data Application Data Management. *Big Data and Cognitive Computing* 6, 2 (May 2022), 49. doi:10.3390/bdcc6020049
- [10] Mat Keep and Henrik Ingo. 2020. Performance Best Practices: Benchmarking. MongoDB Blog. Retrieved 2025-10-01 from <https://www.mongodb.com/company/blog/performance-best-practices-benchmarking> Updated April 20, 2020.
- [11] MongoDB, Inc. 2025. *MongoDB Manual: Data Model Design*. Retrieved 2025-10-02 from <https://www.mongodb.com/docs/manual/data-model-design/>
- [12] MongoDB, Inc. 2025. *MongoDB Manual: Indexes*. Retrieved 2025-10-02 from <https://www.mongodb.com/docs/manual/indexes/>
- [13] PostgreSQL Global Development Group. 2025. *PostgreSQL Documentation: Multicolumn Indexes*. Retrieved 2025-10-02 from <https://www.postgresql.org/docs/current/indexes-multicolumn.html>
- [14] PostgreSQL Global Development Group. 2025. *PostgreSQL Documentation: Table Constraints*. Retrieved 2025-10-02 from <https://www.postgresql.org/docs/current/ddl-constraints.html>
- [15] Mark Raasveldt, Pedro Holanda, Tim Gubner, and Hannes Mühleisen. 2018. Fair Benchmarking Considered Difficult: Common Pitfalls in Database Performance Testing. In *DBTest'18: Workshop on Testing Database Systems* (Houston, TX, USA). Association for Computing Machinery, New York, NY, USA, 6 pages. doi:10.1145/3209950.3209955
- [16] Sanket Vilas Salunke and Abdelkader Ouda. 2024. A Performance Benchmark for the PostgreSQL and MySQL Databases. *Future Internet* 16, 10 (Oct. 2024), 382. doi:10.3390/fi16100382
- [17] William Schultz and Murat Demirbas. 2025. Design and Modular Verification of Distributed Transactions in MongoDB. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5045–5058. doi:10.14778/3750601.3750626
- [18] Toni Taipalus. 2024. Database management system performance comparisons: A systematic literature review. *Journal of Systems and Software* 208 (Feb. 2024), 111872. doi:10.1016/j.jss.2023.111872